УДК XXX.XXX

СРАВНЕНИЕ ПЛАТФОРМЕННЫХ ПОТОКОВ, ВИРТУАЛЬНЫХ ПОТОКОВ И РЕАКТИВНОГО ПОДХОДА В JAVA-СЕРВИСАХ С ОЖИДАНИЕМ ВВОДА-ВЫВОДА

Дощенников Н. А.[1] (бакалавр)
Научный руководитель – доктор технических наук, старший преподаватель
Береснев А. Д.[1]
[1]Университет ИТМО
xxx@yandex.ru

Аннотация
Рассматривается организация конкурентной обработки запросов в веб-сервисах на платформе Java, которые значительную часть времени ожидают ответов внешних систем, например баз данных. Основное внимание уделено сравнению трёх подходов, то есть классических платформенных потоков, виртуальных потоков, появившихся в Java 21, и реактивного программирования на примере Spring WebFlux. Статья изучает, как выбор подхода влияет на производительность сервиса при росте числа одновременных клиентов, при увеличении времени ожидания внешнего сервиса, при перегрузке и при ограниченном объёме памяти. Для этого проводится нагрузочное тестирование на стенде с имитацией внешнего сервиса, а подходы сопоставляются по количественным метрикам, а именно по числу успешных ответов в секунду, времени ответа, загрузке процессора, паузам сборщика мусора и минимальному размеру кучи, при котором сервис сохраняет работоспособность. Результаты представляют интерес для разработчиков и архитекторов, выбирающих модель конкурентности для нового или переносимого сервиса, особенно в условиях высокой нагрузки и ограниченных ресурсов, а также помогают оценить, когда достаточно виртуальных потоков и когда оправдан переход на реактивную модель. Выводы относятся к сервисам с преобладанием операций ввода-вывода.
Ключевые слова
Виртуальные потоки, реактивное программирование, Spring WebFlux, Java, пропускная способность, сборщик мусора, нагрузочное тестирование.

Введение

Сервисы, основную часть времени ожидающие ответа базы данных или другой внешней системы, традиционно строятся по модели «один запрос, один поток», и число одновременно обрабатываемых запросов в ней ограничено размером пула потоков операционной системы. Известны два способа снять это ограничение. Первый основан на реактивном программировании, например на Spring WebFlux, который обслуживает тысячи запросов малым числом потоков, но требует переписывания кода в функциональном стиле. Второй способ связан с виртуальными потоками, окончательно включёнными в Java 21 [1]. Они сохраняют привычный последовательный код, а ожидание ввода-вывода не занимает поток операционной системы. Сравнению виртуальных потоков с платформенными посвящены, в частности, работы Lašić и соавторов [2] и Максимова [3]. Первые авторы исследуют эффективность виртуальных потоков в серверном приложении, работающем с разными системами управления базами данных, и показывают, что выигрыш зависит от конкретной системы и характера операций. Максимов сопоставляет платформенные и виртуальные потоки, а также корутины Kotlin на синтетической задаче записи в файл при возрастающем числе параллельных задач и отмечает преимущество легковесных потоков по задержке и потреблению памяти. Однако эти работы не рассматривают реактивный подход. Прямое сравнение виртуальных потоков со Spring WebFlux выполнено в работе Brzeziński и соавторов [4]. Авторы оценивают оба подхода на сервисе с базой данных при разных конфигурациях вычислительных ресурсов и приходят к выводу, что виртуальные потоки выигрывают по задержке и пропускной способности, а реактивный подход по

потреблению памяти. Этот результат трудно перенести на практику по нескольким причинам. В работе нет платформенных потоков в качестве базы сравнения, использована только Java 21 с её особенностями работы виртуальных потоков, нагрузка не превышала 1000 одновременных запросов, а реализации различались слоем доступа к данным, то есть JDBC и R2DBC, поэтому разницу нельзя целиком отнести к модели потоков. В настоящей работе сравниваются три стека целиком, а именно Spring MVC с пулом платформенных потоков, тот же код с виртуальными потоками и Spring WebFlux. Измерения выполнены в единых условиях на Java 25, где устранено закрепление виртуальных потоков при синхронизации [5]. Внешний сервис имитируется с заданной задержкой, что отделяет влияние модели потоков от особенностей конкретной базы данных. Нагрузка доходит до 10 000 одновременных клиентов и создаётся в закрытой и открытой моделях, а помимо пропускной способности и задержки оцениваются нагрузка на процессор, паузы сборщика мусора и минимальный размер кучи.

Цель работы состоит в том, чтобы определить, при каком числе одновременных запросов и каких ограничениях по памяти виртуальные потоки обеспечивают ту же производительность, что и реактивный подход, и в каких случаях платформенные потоки оказываются непригодными. Далее описана методика экспериментов, приведены и интерпретированы результаты, после чего обсуждаются практические выводы и риски внедрения.

Методика

Объектом исследования служили три способа организации конкурентной обработки запросов в Java-сервисе, основную часть времени ожидающем ответа внешней системы. Первый способ представляет собой классическую модель Spring MVC на сервере Tomcat версии 10.1, в которой каждый запрос обрабатывается отдельным потоком операционной системы из пула фиксированного размера. В экспериментах размер пула равнялся 200 потокам, что соответствует значению Tomcat по умолчанию. Второй способ использует то же самое приложение, но с включёнными виртуальными потоками [1], так что каждый запрос выполняется в новом виртуальном потоке. Третий способ реализован на Spring WebFlux поверх Netty и опирается на неблокирующую модель с небольшим числом потоков цикла событий. Первые два варианта используют один и тот же программный код и различаются только конфигурацией, поэтому разницу в их поведении можно связывать именно с типом потоков. Следует подчеркнуть, что в работе сравниваются стеки целиком, а не изолированные механизмы потоков. Все приложения собраны на Spring Boot 3.5.0. Для обращения к внешнему сервису блокирующие варианты применяли клиент Apache HttpClient 5, а реактивный вариант клиент WebClient. Размер пула соединений к внешнему сервису во всех вариантах равнялся 10 000, чтобы он не ограничивал пропускную способность.

На каждый запрос сервис обращался к имитируемому внешнему сервису, который отвечал через заданное время и возвращал небольшой объект. В части экспериментов за один запрос выполнялись три таких обращения подряд. Имитатор реализован на WebFlux и Netty и не занимает поток на время ожидания, поэтому тысячи одновременных ожиданий почти не влияют на результаты. Данное обстоятельство учитывалось при интерпретации значений вблизи верхней границы измеренной пропускной способности.

Все эксперименты проведены на одной машине с процессором Ryzen 7 5700U, имеющим восемь физических ядер и шестнадцать потоков, и 16 ГБ оперативной памяти под управлением Linux. Тестируемое приложение, имитатор внешнего сервиса и генератор нагрузки k6 версии 2.3.0 работали как отдельные процессы и были закреплены за непересекающимися группами физических ядер, соответственно четырьмя, двумя и двумя. Взаимодействие происходило через интерфейс локальной петли. Использовалась Java 25.0.5, выбранная как версия с долгосрочной поддержкой, и сборщик мусора G1, применяемый по умолчанию. Начальный и максимальный размер кучи приложения были

равны и составляли 2 ГБ, за исключением отдельного эксперимента по влиянию размера кучи. Куча имитатора составляла 1 ГБ.

Нагрузка создавалась в закрытой и открытой моделях. В закрытой модели фиксированное число клиентов, от 200 до 10 000, непрерывно отправляет запросы, и каждый следующий запрос посылается только после получения ответа на предыдущий. Если сервис отвечает медленнее, клиенты автоматически снижают интенсивность запросов. Максимальное время ожидания ответа в этой модели равнялось значению k6 по умолчанию, то есть 60 с. В открытой модели запросы поступают с заданной частотой, от 1000 до 6000 в секунду, независимо от того, успел ли сервис ответить на предыдущие. При нехватке мощности запросы накапливаются, как это происходит в реальном потоке пользователей. Запрос, не завершившийся за 10 с, считался неуспешным. Число одновременно активных клиентов генератора было ограничено 20 000, и запросы, которые не удалось отправить из-за этого ограничения, учитывались отдельно.

Использовались два основных сценария, различающихся задержкой имитатора, которая составляла 100 мс для быстрого и 1000 мс для медленного внешнего сервиса. Дополнительно были выполнены эксперименты с пулами Tomcat размером от 50 до 1000 потоков, с задержкой 5 с и с цепочкой из трёх последовательных обращений к внешнему сервису. Влияние объёма памяти оценивалось при 10 000 клиентов и задержке 1000 мс для размеров кучи 256 МБ, 512 МБ, 1 ГБ, 2 ГБ и 4 ГБ. Минимальной считалась наименьшая куча, при которой приложение завершало замер без ошибки нехватки памяти и сохраняло способность отвечать на запросы.

Для каждого варианта измерялось число успешных ответов в секунду, которое вычислялось как пропускная способность по данным k6, умноженная на долю успешных запросов. Также определялся 95-й процентиль времени ответа, то есть значение, которое не превышают 95% запросов. В него входят и запросы, завершившиеся по превышении времени ожидания. Нагрузка на процессор оценивалась как среднее число ядер, занятых процессом приложения, по приращению затраченного процессорного времени за время замера. Влияние сборщика мусора характеризовалось суммарной длительностью и числом пауз за время замера, которые определялись по журналу самой виртуальной машины. Дополнительно фиксировались время запуска приложения, задержка первого запроса после запуска и объём памяти, занятый процессом сразу после старта.

Все измерения выполнялись по единой процедуре. Для каждого сочетания варианта и уровня нагрузки приложение запускалось заново на холодной виртуальной машине и в течение 20 с прогревалось под нагрузкой того же уровня, причём результаты прогрева не учитывались. После этого выполнялись три замера по 45 с с паузой 5 с между ними. В экспериментах с размером кучи и в открытой модели выполнялось по два замера, поэтому в этих случаях приводится среднее двух значений, а в остальных случаях медиана трёх.

Предложенная методика имеет ряд ограничений. Приложение, имитатор и генератор нагрузки работают на одной машине и делят кэш, память и тепловой бюджет мобильного процессора, даже будучи закреплёнными за разными ядрами. Имитатор лишь задаёт время ответа и не воспроизводит особенностей реальной сетевой службы или базы данных. В ходе экспериментов пропускная способность достигала примерно 16 000 запросов в секунду, и выше этого значения результат мог определяться процессорными ресурсами стенда, в частности четырьмя ядрами, выделенными приложению, а не свойствами самого подхода. Наконец, полученные выводы относятся к одной версии Java. Начиная с Java 24, синхронизированные блоки перестали закреплять виртуальный поток за потоком операционной системы [5], поэтому на более ранних версиях поведение виртуальных потоков может отличаться.

Результаты

Зависимость пропускной способности от числа клиентов при медленном внешнем сервисе показана на рис. 1. Для платформенных потоков она практически не меняется и

составляет от 197 до 199 запросов в секунду. Это значение совпадает с отношением размера пула к времени ожидания внешнего сервиса, то есть 200 потоков, делённых на 1 с. Дополнительный эксперимент с пулами другого размера подтвердил такую связь. При 2000 клиентов пулы в 50, 100, 200, 500 и 1000 потоков обеспечили соответственно 49, 100, 199, 497 и 994 запроса в секунду. Таким образом, потоки, заблокированные в ожидании ответа, целиком определяют потолок пропускной способности, а добавление клиентов его не повышает. Виртуальные потоки и реактивный вариант, напротив, следуют теоретическому пределу, равному числу клиентов, делённому на время ожидания. До 2000 клиентов их результаты совпадают с точностью до долей процента, а при 5000 клиентов различие составляет около 1%. Расхождение появляется только при 10 000 клиентов, где реактивный вариант обрабатывает 8777 запросов в секунду, а виртуальные потоки 7525, то есть на 14% меньше. Процесс с виртуальными потоками при этом занимал в среднем 5,6 ядра против 2,9 у реактивного варианта, а суммарная длительность пауз сборщика мусора за замер составила 19,9 с против 5,2 с. Следовательно, при очень большом числе одновременных запросов ограничением для виртуальных потоков становится уже не число потоков, а нагрузка на память и сборщик мусора.

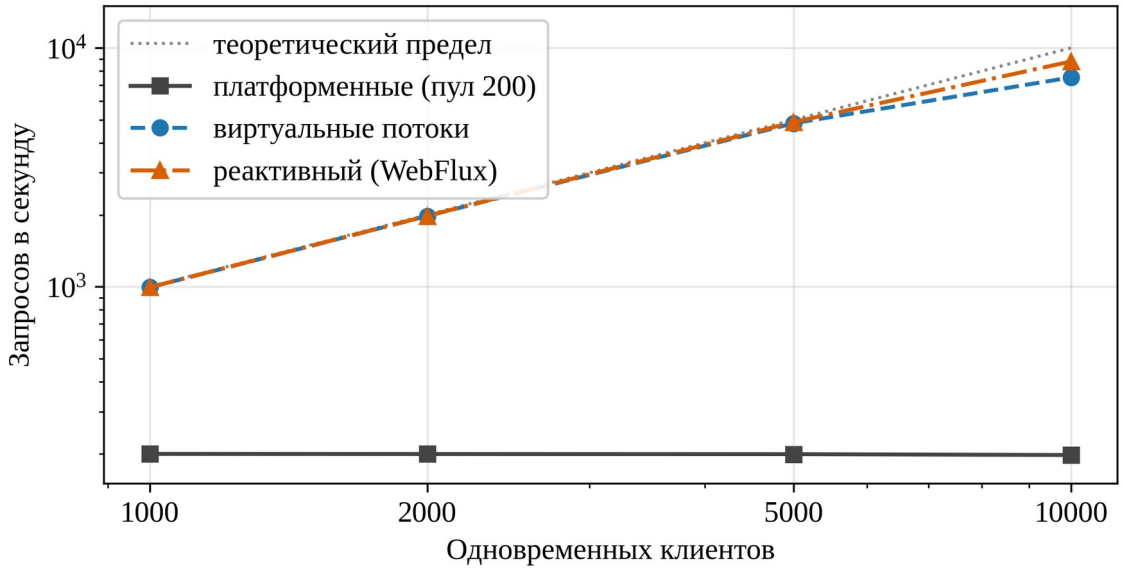


Рис. 1. Пропускная способность при задержке внешнего сервиса 1000 мс

Задержка при тех же условиях показана на рис. 2. У платформенных потоков 95-й процентиль времени ответа растёт пропорционально числу клиентов и равен 5,0 с при 1000 клиентов, 10,0 с при 2000, 25,0 с при 5000 и 50,1 с при 10 000. Эти значения почти точно равны числу клиентов, делённому на пропускную способность, как того требует закон Литтла [1]. Физически это означает, что почти всё время запрос проводит в очереди к свободному потоку, а не в ожидании внешнего сервиса, которое занимает всего 1 с. При 10 000 клиентов значение превышает длительность самого замера, поскольку k6 дожидается завершения начатых запросов. У виртуальных потоков и реактивного варианта задержка остаётся близкой к времени ожидания внешнего сервиса вплоть до 5000 клиентов, где она составляет 1,05 и 1,04 с. При 10 000 клиентов она возрастает до 1,86 с у виртуальных потоков и до 1,33 с у реактивного варианта. Рост задержки происходит при тех же условиях, что и падение пропускной способности на рис. 1, и в обоих случаях хуже оказываются виртуальные потоки.

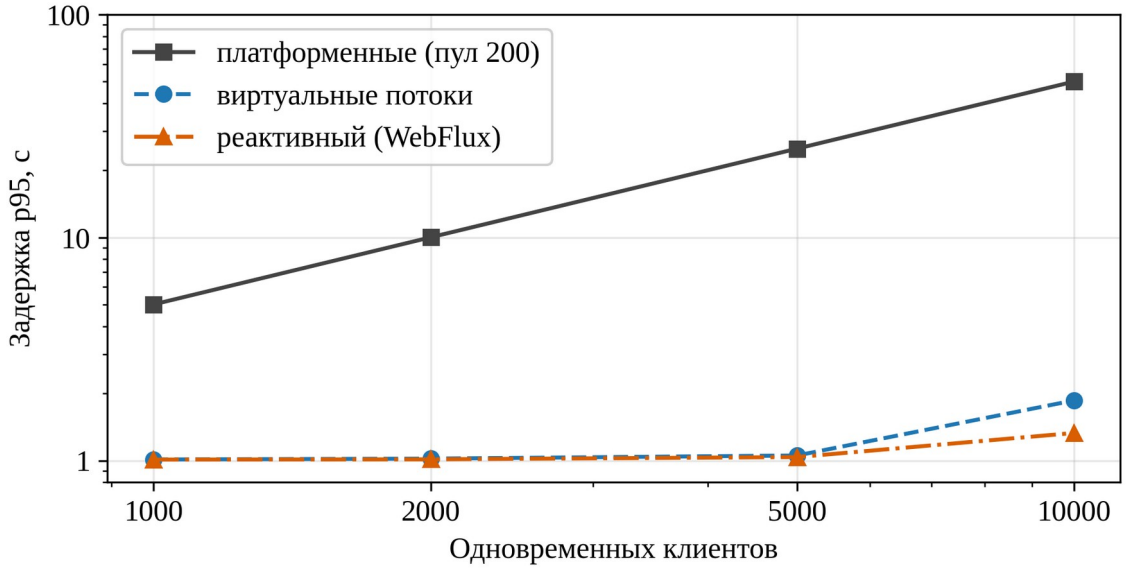


Рис. 2. Задержка (95-й процентиль) при задержке внешнего сервиса 1000 мс

Влияние объёма памяти при 10 000 клиентов и задержке 1000 мс показано на рис. 3. Платформенные потоки работали при любом размере кучи, включая 256 МБ, однако ценой этого остаются те же 197 запросов в секунду, поэтому на рисунке их столбики почти не видны. Реактивный вариант не выдержал нагрузку при 256 МБ и завершился ошибкой нехватки памяти, но уже при 512 МБ обрабатывал 8521 запрос в секунду. При дальнейшем увеличении кучи его результат почти не менялся и составил 8744, 8665 и 8908 запросов в секунду для 1 ГБ, 2 ГБ и 4 ГБ. Следует отметить, что при 512 МБ суммарная длительность пауз сборщика мусора достигала 12,1 с за замер, так что запас по памяти был невелик. Виртуальные потоки завершились ошибкой нехватки памяти при 256 МБ и 512 МБ, а при 1 ГБ приложение перестало отвечать на запросы. Работоспособным этот вариант стал только при 2 ГБ, где пропускная способность составила 7346 запросов в секунду, что на 15% ниже, чем у реактивного варианта при такой же куче. Увеличение кучи до 4 ГБ сократило отставание до 7%, а суммарную длительность пауз сборщика мусора уменьшило с 19,1 с до 8,7 с. Эти данные показывают, что за простой блокирующий код при виртуальных потоках приходится платить памятью. Косвенно это подтверждает занятость кучи сразу после сборки мусора. В сценарии с медленным внешним сервисом при 2000 и 5000 клиентов её максимум составлял у реактивного варианта 105 и 195 МБ, а у виртуальных потоков 773 и 1869 МБ при практически одинаковой пропускной способности. Эта величина включает и мусор, ещё не убранный из старших поколений, поэтому служит лишь оценкой сверху для объёма живых данных. Вероятное объяснение состоит в том, что каждый из одновременно ожидающих запросов удерживает в памяти собственный стек вызовов и связанные с ним объекты, тогда как в реактивном варианте хранится лишь компактное описание состояния запроса. Это предположение, так как структура занятой памяти напрямую не исследовалась.

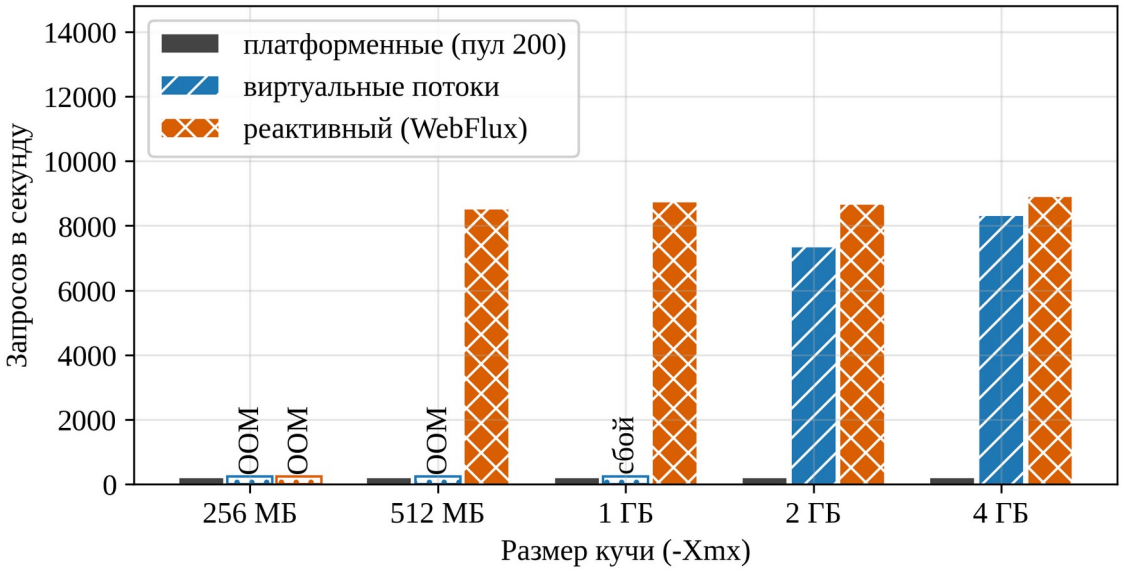


Рис. 3. Пропускная способность при 10 000 клиентов в зависимости от размера кучи

Поведение при перегрузке в открытой модели показано на рис. 4. Пока поступающая нагрузка не превышает возможностей сервиса, все три варианта обрабатывают столько запросов, сколько поступает. Для платформенных потоков предельное значение равно 200 потокам, делённым на 100 мс, то есть 2000 запросов в секунду. При нагрузке 1900 запросов в секунду они ещё справляются со временем ответа около 100 мс. Уже при 3000 запросов в секунду пропускная способность остаётся на уровне 1972, а 95-й процентиль возрастает до 6,5 с, так как запросы накапливаются в очереди, и около 24% запросов генератор нагрузки не смог отправить, поскольку все его клиенты были заняты ожиданием. При 6000 запросов в секунду не были отправлены уже около 60% запросов, а из отправленных 30% завершились превышением времени ожидания. Число успешных ответов упало до 1378 в секунду, что составляет лишь около 23% от поступающей нагрузки и меньше, чем при 3000 запросов в секунду. Вероятно, сервис продолжает тратить потоки на запросы, клиенты которых уже отказались ждать, поэтому полезная работа сокращается по мере роста перегрузки. Виртуальные потоки и реактивный вариант обработали всю нагрузку вплоть до 6000 запросов в секунду без единой ошибки. Их 95-й процентиль составил около 101 мс при 3000 запросов в секунду и 156 мс при 6000.

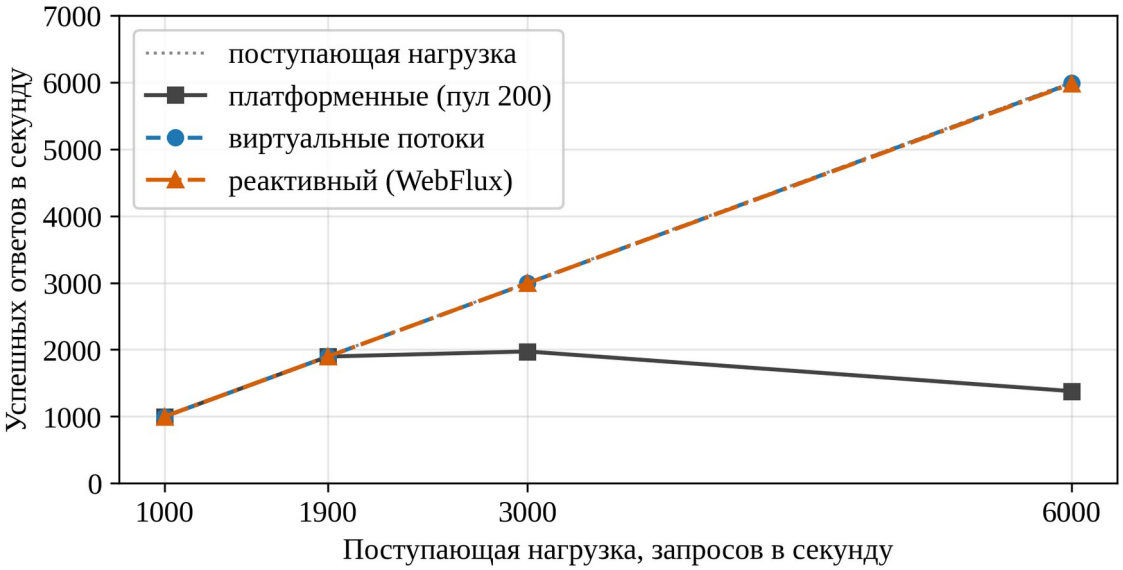


Рис. 4. Успешные ответы при перегрузке в открытой модели

При быстром внешнем сервисе с задержкой 100 мс картина в основном повторяется, но различия между виртуальными потоками и реактивным вариантом проявляются раньше. Платформенные потоки по-прежнему ограничены отношением размера пула ко времени ожидания, то есть 200 потоков, делённых на 0,1 с, и обрабатывают около 1960 запросов в секунду независимо от числа клиентов от 200 до 5000. Виртуальные потоки и реактивный вариант до 1000 клиентов совпадают с точностью до долей процента, обрабатывая 9429 и 9444 запроса в секунду при 95-м процентиле около 136 мс. При 2000 клиентов различие составляет около 5%, при этом реактивный вариант достигает 15 582 запросов в секунду, а виртуальные потоки 14 881. Рост числа клиентов до 5000 почти не увеличивает результат реактивного варианта, который составляет 16 143 запроса в секунду, и это указывает на достижение предела стенда. Процесс при этом занимает в среднем около 4,3 ядра. У виртуальных потоков при 5000 клиентов пропускная способность падает до 13 914 запросов в секунду, то есть на 14% ниже, а 99-й процентиль времени ответа достигает 1,3 с против 0,6 с у реактивного варианта. Суммарная длительность пауз сборщика мусора за замер при этом составляет 10,6 с против 0,7 с, а средняя загрузка процессора 5,9 ядра против 4,3. Рост отставания виртуальных потоков совпадает с ростом суммарной длительности пауз сборщика мусора, что указывает на его роль в этих потерях.

В сценарии с тремя последовательными обращениями к внешнему сервису по 100 мс потолок платформенных потоков равен 200 потокам, делённым на 0,3 с, то есть около 667 запросов в секунду. Измеренные 657 запросов в секунду при 500 и при 2000 клиентов с ним согласуются. При 500 клиентов виртуальные потоки и реактивный вариант практически не различаются, обрабатывая 1638 и 1643 запроса в секунду при 95-м процентиле около 310 мс. При 2000 клиентов оба варианта многократно превосходят платформенные потоки, причём виртуальные потоки оказались впереди, обработав 6240 запросов против 5822 у реактивного варианта, то есть на 7% больше, при задержке 375 мс против 449 мс. Разброс между повторами при этом составил около 270 и 430 запросов в секунду, поэтому различие умеренное, хотя интервалы значений почти не пересекаются. Следовательно, преимущество реактивного подхода, наблюдавшееся при 10 000 клиентов, не является универсальным и зависит от сценария.

Холодный старт оказался сопоставим у всех вариантов. Приложение запускалось за 2,4–3,1 с, причём медианы составили 2,6 с у платформенных и виртуальных потоков и 2,7 с у реактивного варианта. Первый запрос после запуска обрабатывался дольше установившегося на 135 мс у платформенных потоков, на 143 мс у виртуальных и на 216 мс у реактивного варианта, что связано с инициализацией классов и неблокирующего клиента. Память процесса сразу после старта составляла в среднем 342, 348 и 381 МБ соответственно. Таким образом, заметных преимуществ или недостатков в запуске ни один из подходов не имеет.

Обсуждение

Полученные результаты позволяют сформулировать ряд практических выводов для выбора стека сервиса, который большую часть времени ожидает ответы внешних систем. Прежде всего, платформенные потоки с пулом фиксированного размера оказываются непригодными, когда число одновременных запросов существенно превышает размер пула. Пропускная способность в этом случае равна отношению размера пула к времени ожидания, а задержка растёт пропорционально числу клиентов. Увеличение пула частично решает проблему, так как пул в 1000 потоков обеспечил 994 запроса в секунду при 2000 клиентов. Однако он потребовал около тысячи потоков операционной системы и лишь отодвигает предел, а не устраняет его.

Виртуальные потоки снимают главное ограничение платформенной модели без отказа от привычного последовательного кода [1]. До 5000 одновременных клиентов при ожидании 1 с и до 1000 клиентов при ожидании 100 мс они не уступали реактивному варианту ни по пропускной способности, ни по задержке. Поскольку число

одновременно обрабатываемых запросов равно произведению их интенсивности на время обработки, такой диапазон соответствует нагрузке порядка нескольких тысяч запросов в секунду, чего достаточно для значительной части прикладных сервисов. Для них переход на виртуальные потоки сводится к изменению конфигурации и не требует переписывания кода.

Реактивный подход сохраняет преимущество в более тяжёлых режимах. При 10 000 одновременных клиентов он обработал на 14% больше запросов, занимал примерно вдвое меньше процессорного времени и укладывался в 512 МБ кучи, тогда как виртуальным потокам потребовалось не менее 2 ГБ. При быстром внешнем сервисе и 5000 клиентов 99-й процентиль времени ответа у виртуальных потоков оказался вдвое выше из-за пауз сборщика мусора. Следовательно, если сервис ограничен по памяти, должен выдерживать десятки тысяч одновременных запросов или чувствителен к редким долгим ответам, реактивная модель остаётся оправданной, несмотря на большую сложность кода. Эти выводы получены для сборщика G1, и с другими сборщиками мусора разрыв может оказаться иным.

При перегрузке виртуальные потоки и реактивный вариант не показали деградации вплоть до 6000 запросов в секунду, тогда как платформенные потоки теряли заметную часть полезной работы. Тем не менее неограниченное число виртуальных потоков не означает неограниченных ресурсов. Как показал эксперимент с памятью, каждый одновременный запрос занимает память, поэтому в промышленной системе число одновременно обрабатываемых запросов стоит ограничивать явно, например с помощью семафора [1].

Применение виртуальных потоков связано с рядом рисков, которые данный эксперимент не охватывает. Во-первых, имитируемый внешний сервис допускал 10 000 одновременных соединений, тогда как реальные системы обычно обращаются к базам данных через пул из нескольких десятков соединений. Виртуальные потоки не ускоряют работу такого ресурса, а лишь переносят очередь с потоков на пул соединений. При этом драйверы JDBC являются блокирующими, и виртуальные потоки позволяют их сохранить, тогда как реактивная альтернатива R2DBC требует переписать слой доступа к данным. Во-вторых, до Java 24 блокирующая операция внутри синхронизированного блока закрепляла виртуальный поток за потоком операционной системы, что могло лишить его преимуществ при использовании библиотек с такой синхронизацией. Начиная с Java 24 эта проблема устранена для синхронизированных блоков, но сохраняется, например, при вызове нативного кода [5]. В-третьих, каждый виртуальный поток имеет собственные значения ThreadLocal, поэтому приёмы, рассчитанные на небольшое число долгоживущих потоков, например кэширование дорогих объектов в ThreadLocal, при большом числе короткоживущих потоков становятся неэффективными и расходуют память [1]. Наконец, все выводы получены на одной машине с имитацией внешнего сервиса, поэтому перед внедрением их стоит проверить на реальной нагрузке.

Заключение

Проведённое сравнение показало, что платформенные потоки с пулом фиксированного размера ограничивают пропускную способность значением, равным отношению размера пула к времени ожидания, то есть около 200 запросов в секунду при задержке внешнего сервиса 1 с, и при нагрузке 6000 запросов в секунду в открытой модели обслуживают лишь около 23% запросов. Виртуальные потоки и реактивный вариант при 5000 одновременных клиентов различаются примерно на 1%, поэтому для значительной части сервисов достаточно перейти на виртуальные потоки без переписывания кода. При 10 000 клиентов реактивный вариант обрабатывает на 14% больше запросов и работает при куче 512 МБ, тогда как виртуальным потокам требуется не менее 2 ГБ. Результаты ограничены одной машиной, имитацией внешнего сервиса, сборщиком G1 и одной версией Java, поэтому их следует рассматривать как оценку соотношений между подходами, а не абсолютных значений. Дальнейшие исследования

целесообразно направить на работу с реальной базой данных при ограниченном пуле соединений, на другие сборщики мусора, в частности на поколенческий ZGC, а также на задачи с интенсивным использованием процессора и на распределённый стенд.

Литература

1. JEP 444: Virtual Threads [Электронный ресурс]. – Режим доступа: https://openjdk.org/jeps/444 (дата обращения 25.10.2026).
2. Lašić L., Beronić D., Mihaljević B., Radovan A. Assessing the Efficiency of Java Virtual Threads in Database-Driven Server Applications // Proc. 47th MIPRO ICT and Electronics Convention (MIPRO). IEEE, 2024. P. 2045–2050. https://doi.org/10.1109/MIPRO60963.2024.10569754.
3. Максимов В. Ю. Прогрессивный параллелизм на практике: от классических потоков к виртуальным в Java и корутинам в Kotlin // Актуальные исследования. 2024. № 24 (206). С. 42–47. https://doi.org/10.5281/zenodo.11526785.
4. Brzeziński J., Charlak D., Kozieł G. Comparative Analysis of Reactive Programming and Java Virtual Threads // Journal of Computer Sciences Institute. 2026. Vol. 39. P. 154–160.
5. JEP 491: Synchronize Virtual Threads without Pinning [Электронный ресурс]. – Режим доступа: https://openjdk.org/jeps/491 (дата обращения 26.10.2026).